CSE 144 Final Project Report

Transfer Learning Challenge

UC Santa Cruz | Spring 2026

Prithvi Poddar - 2140913

1 Introduction

This project addresses a 100-class image classification challenge using transfer learning on a limited dataset sampled from multiple source domains. Each class contains only 10 training images, making the task a few-shot learning problem where standard training from scratch would overfit severely.

Transfer learning is highly appropriate here because deep convolutional networks pretrained on ImageNet have already learned rich, general-purpose visual features such as edges, textures, and object parts. By fine-tuning these features on our small dataset rather than training from scratch, we benefit from millions of training examples the backbone has already seen.

My approach fine-tunes an EfficientNet-B2 backbone pretrained on ImageNet-1K, with aggressive data augmentation and label smoothing to combat overfitting. The final model achieved a Kaggle test set accuracy of 62.7%, exceeding the required 60% baseline.

2 Dataset

Property	
Number of classes	100
Training images	1,079 (~10 per class)
Validation images	~215 (20% split)
Test images	1,000 (unlabeled)
Image format	Colored JPEGs, variable sizes
Input size after resize	224 x 224 pixels

The training directory is organized as train/<class_id>/ where each folder name is a string integer from 0 to 99. The label mapping is identity: folder "0" maps to Label 0, folder "1" to Label 1, and so on. This ordering is enforced by sorting folder names numerically before assigning labels.

Preprocessing: All images are resized to 224x224 and normalized using ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]).

Data augmentation (training only): random crop from 256x256, random horizontal flip, random vertical flip (p=0.1), color jitter (brightness, contrast, saturation, hue), random rotation up to 15 degrees, and random grayscale (p=0.05).

3 Implementation

3.1 Model

I selected EfficientNet-B2 as the pretrained backbone, loaded with ImageNet-1K weights from TorchVision. EfficientNet-B2 was chosen for its strong accuracy-to-parameter ratio (~9M parameters), making it well-suited to a small dataset where larger models such as ResNet-101 or ViT would risk overfitting.

The original classifier head was replaced with a custom head consisting of a Dropout layer (p=0.4) followed by a single Linear layer mapping to 100 output classes. The rest of the network was left unchanged and fully fine-tuned from the start.

No layer freezing was applied. All parameters were trained end-to-end from epoch 1, which empirically performed better than a two-phase freeze/unfreeze strategy given the relatively short training run.

3.2 Training

- Loss function: Cross-entropy with label smoothing (epsilon=0.1) to reduce overconfident predictions on the small dataset.
- Optimizer: AdamW with learning rate 1e-3 and weight decay 1e-4.
- Scheduler: OneCycleLR with 10% linear warm-up followed by cosine annealing, running for 30 epochs.
- Batch size: 32.
- Epochs: 30.
- Hardware: Google Colab free tier, NVIDIA T4 GPU, PyTorch 2.x, Python 3.12.
- Training time: approximately 15-20 minutes on T4 GPU.

4 Experiments

Baseline: EfficientNet-B2 with ImageNet weights, 30 epochs, lr=1e-3, batch size 32, standard augmentation (resize + horizontal flip only). This served as the starting configuration.

Hyperparameter tuning was performed manually by monitoring validation accuracy across runs. The 80/20 train/val split was used for all validation, with a fixed random seed (42) for reproducibility.

Key ablations I explored:

- Augmentation strength: Adding color jitter, rotation, and vertical flip improved val accuracy by approximately 4-5% compared to minimal augmentation.
- Label smoothing: Smoothing (epsilon=0.1) reduced overfitting and improved generalization on the small dataset.
- Learning rate: lr=1e-3 with OneCycleLR outperformed constant lr schedules.
- Backbone size: EfficientNet-B2 outperformed EfficientNet-B0 slightly while remaining fast enough for Colab.

5 Results

Metric	
Best validation accuracy	60.0%
Final training accuracy (epoch 30)	98.3%
Final validation accuracy (epoch 30)	60.0%
Kaggle public leaderboard score	62.727%
Best model saved at epoch	25 / 30

Training accuracy climbed steadily from ~1.6% at epoch 1 to ~98% by epoch 30, while validation accuracy improved from ~3.7% to a peak of 60% around epoch 25. The large gap between training and validation accuracy in later epochs indicates overfitting, which is expected given only ~8-9 training images per class after the validation split.

The Kaggle public leaderboard score of 62.727% (evaluated on ~10% of the test set) slightly exceeded validation accuracy, suggesting the model generalizes reasonably to unseen data.

6 Discussion

The most impactful decisions were the choice of a pretrained EfficientNet-B2 backbone and strong data augmentation. Without pretraining, achieving above-chance accuracy on 100 classes with only 10 training images each would be nearly impossible. Augmentation effectively expanded the training set and reduced overfitting.

The main failure mode is overfitting: by epoch 25+, training accuracy exceeds 98% while validation accuracy plateaus around 60%, indicating the model memorizes training examples. Classes that are visually similar to each other (sampled from different datasets) are likely the primary source of errors.

Concrete next improvements would include: (1) test-time augmentation (TTA) to ensemble multiple augmented views of each test image, (2) ensembling predictions

from multiple backbone architectures such as EfficientNet-B2 and ConvNeXt-Tiny, and (3) using a lower learning rate for longer training (e.g., lr=3e-4 for 50 epochs) to allow more careful convergence.

7 Reproducibility

All experiments use a fixed random seed of 42, applied to Python, NumPy, PyTorch CPU, and PyTorch CUDA via torch.backends.cudnn.deterministic=True.

Environment:

- Python 3.12
- PyTorch 2.x (Colab default)
- torchvision (Colab default)
- pandas, pillow

To reproduce training:

```
python train.py --mode train --data_dir data/ --output_dir outputs/ --epochs
30 --backbone efficientnet_b2 --seed 42
```

To generate submission.csv:

```
python train.py --mode test --data_dir data/ --weights outputs/best_model.pth
--output_dir outputs/
```

8 Team Contributions

- Prithvi Poddar: All work including data preparation, model design, training, hyperparameter tuning, inference, and report writing.

9 References

- TorchVision Models and Pretrained Weights: https://pytorch.org/vision/stable/models.html
- Tan, M. & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML 2019.
- Loshchilov, I. & Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR 2019.
- Smith, L. (2018). A disciplined approach to neural network hyper-parameters. arXiv:1803.09820.